Plan: Align 2025 pm_functions.R to Match orig (Gold Standard)

pmsimstats team

2026-03-24

Contents

1 Objective **1**

2 Scope **1**

3 Naming Convention **1**

4 Step 1: Replace mod_gompertz with modgompertz_orig logic **1**

5 Step 2: Fix column naming in generate_data / build_sigma_matrix **2**

6 Step 3: Fix the category labels in parameter tables **3**

7 Step 4: Rewrite lme_analysis to match orig's formula logic **3**

8 Step 5: Match MVN draw method **4**

9 Step 6: Match PD handling in build_sigma_matrix **5**

10 Step 7: Add missing infrastructure functions **6**
10.1 7a: build_trial_design (from buildtrialdesign) . . . 6
10.2 7b: censor_data (from censordata) . . . 6
10.3 7c: generate_simulated_results (from generateSimulatedResults) . . . 6

11 Step 8: Style Reconciliation **6**
11.1 Function name mapping . . . 6
11.2 Variable name mapping (inside functions) . . . 6
11.3 Style rules . . . 6
11.4 Estimated style-only changes . . . 7

12 Step 9: Remove dead code from 2025 **7**

13 Execution Order and Dependencies **8**

14 Estimated Effort **9**

15 Validation Strategy **9**

1 Objective

Make the 2025 tidyverse simulation pipeline produce statistically identical results to the orig data.table pipeline for identical parameter inputs. The 2025 codebase should be a clean, contemporary R implementation of the same simulation – differing only in style (tidyverse, snake_case, tibble), not in substance.

2 Scope

The target file is `analysis/2025/pm_functions.R` (to be created in pmsimstats-ng). The orig functions in `R/` are frozen as the gold standard. Changes flow in one direction: 2025 adapts to orig.

3 Naming Convention

The 2025 code already uses snake_case. The orig code uses camelCase for exported functions (`buildSigma`, `generateData`, `lme_analysis`) and internal variables. The 2025 versions will keep snake_case throughout. A mapping table is provided in Step 8.

---

4 Step 1: Replace mod_gompertz with modgompertz_orig logic

What changes: Replace `mod_gompertz()` (line 12) with a snake_case version of the orig's `modgompertz()` that passes through the origin.

Current (2025):

```
mod_gompertz <- function(time, max_value, displacement, rate) {
  max_value * (1 - exp(-displacement * exp(-rate * time)))
}
```

Target:

```
mod_gompertz <- function(t, maxr, disp, rate) {
  y <- maxr * exp(-disp * exp(-rate * t))
  vert_offset <- maxr * exp(-disp * exp(-rate * 0))
  y <- y - vert_offset
  y * (maxr / (maxr - vert_offset))
}
```

Justification: The current formula `max*(1 - exp(-disp*exp(-rate*t)))` is mathematically different from the Hendrickson formulation. It does not pass through the origin (f(0) != 0), which shifts every component mean in the MVN distribution. This is the single largest source of divergence between the two codebases. The `modgompertz_orig()` that already exists at line 1363 of pm_functions.R proves this is known.

Difficulty: Trivial. Replace one function body. All callers use the same positional arguments (time, max, disp, rate). The parameter names differ (snake_case vs abbreviated), but the function is called with positional args throughout.

Verification: `mod_gompertz(0, 10, 5, 0.42)` must equal 0. `mod_gompertz(200, 10, 5, 0.42)` must equal 10 (within tolerance).

---

5 Step 2: Fix column naming in generate_data / build_sigma_matrix

What changes: Replace `"biomarker"` and `"baseline"` labels with `"bm"` and `"BL"` in the sigma matrix labels, and replace `"participant_id"` with `"ptID"` in the output data.

Current (2025):

```
labels <- c("biomarker", "baseline", ...)
```

Target:

```
labels <- c("bm", "BL", ...)
```

Also in `process_participant_data`:

```
participant_data <- participant_data %>%
  mutate(ptID = 1:N)
```

Also in `prepare_analysis_data` and `lme_analysis`: all references to `biomarker`, `baseline`, `symptoms`, `participant_id` must map to `bm`, `BL`, outcome columns, `ptID`.

Justification: The column names flow into the correlation matrix row/col names, which flow into the BM-BR correlation assignment (`correlations["bm", name1]`), which flow into the analysis model formula, and ultimately into coefficient extraction. Mismatched names would cause silent failures or NA results.

Difficulty: Moderate. Roughly 40–50 string replacements across generate_data, build_sigma_matrix, build_correlation_matrix, process_participant_data, prepare_analysis_data, and lme_analysis. Must be done atomically – partial changes will break the pipeline.

Verification: `generate_data()` output must have columns named `bm`, `BL`, `ptID`, `OL1`...`OL8` (matching orig).

---

6 Step 3: Fix the category labels in parameter tables

What changes: The 2025 resp_param and baseline_param tables use long category names (`"time_variant"`, `"pharm_biomarker"`, `"bio_response"`, `"biomarker"`, `"baseline"`). The orig uses short names (`"tv"`, `"pb"`, `"br"`, `"bm"`, `"BL"`).

Options:

- (A) Change the parameter tables passed to 2025 functions to use short names. This forces callers to adapt.
- (B) Change the 2025 functions to accept either format via a lookup. Adds complexity.
- (C) Keep long names internal to 2025 but map at boundaries.

Recommendation: Option A. The parameter tables are defined in analysis scripts, not in pm_functions.R. Change the analysis scripts to use orig-compatible parameter tables. Then the functions can use the same indexing as orig (`resp_param[cat=="tv"]$sd`).

Justification: Using the same parameter format eliminates an entire class of mapping bugs and makes it possible to share parameter definitions between the two pipelines.

Difficulty: Easy. The `factor_types` / `factor_abbreviations` parallel arrays can be collapsed to just the abbreviations. About 20 references to long names need updating.

Verification: Parameter tables must be identical in structure to those used by orig's analysis scripts.

---

7 Step 4: Rewrite lme_analysis to match orig's formula logic

What changes: The 2025 `lme_analysis()` always uses `symptoms ~ biomarker + drug_binary + biomarker:drug_binary`. The orig conditionally builds the formula based on design type.

Target (matching orig logic):

1. Prepend a BL row to the trial design; compute cumulative `t`.
2. Melt to long form; merge trial design.
3. Compute `Dbc`:
   - On-drug: `Dbc = 1`
   - Off-drug: `Dbc = (1/2)^(scalefactor * tsd / t1half)`
4. Test `varInDb`: does Db (on-drug indicator) vary within subjects?
5. Test `varInExp`: does expectancy vary across timepoints?
6. Build formula conditionally:
   - Base: `Sx ~ bm + t`
   - If `varInDb`: `+ Dbc + bm:Dbc`
   - Else: `+ bm:t` (and filter to ever-on-drug participants)
   - If `varInExp > 1` and `useDE`: `+ De`
   - If `simplecarryover` and `varintsd`: `+ tsd`
7. Remove NA rows for model variables.

8. Fit `nlme::lme` with `corCAR1`, fallback without.
9. Extract `bm:Dbc` or `Dbc:bm` (crossover) or `bm:t/t:bm` (open label).

Justification: This is the second largest source of divergence. The formula determines which coefficient is tested for significance. For open-label designs, orig tests a time-biomarker interaction (`bm:t`) while 2025 tests a drug-biomarker interaction (`biomarker:drug_binary`). These are substantively different hypotheses. The expectancy term `De` (actual expectancy value) vs `t` (time) also changes model interpretation.

Difficulty: High. This is the most complex change. The orig implementation is ~180 lines of data.table code with eval/parse idioms for column selection. The 2025 rewrite should use tidyverse idioms (pivot_longer, left_join) but replicate the exact conditional logic. Key challenges:

- The `varInDb` test requires computing per-participant mean of Db and checking if it is always 0 or 1. This is straightforward in either framework.
- The eval/parse column selection (`evalstring <- paste(...)`) in orig can be replaced with tidy selection.
- The `Dbc` computation must use `(1/2)^(scalefactor*tsd/t1half)` directly, NOT lag-based propagation.

Verification: For identical simulated data, the two lme_analysis functions must return identical beta, betaSE, and p-value (within floating-point tolerance). Test on OL, CO, and Hybrid designs.

---

8 Step 5: Match MVN draw method

What changes: Replace `MASS::mvrnorm()` with Cholesky-based draw matching orig.

Current (2025):

```
participant_data <- MASS::mvrnorm(
  n = model_param$N, mu = means, Sigma = sigma,
  empirical = empirical)
```

Target:

```
chol_sigma <- chol(sigma)
Z <- matrix(rnorm(n * p), nrow = n, ncol = p)
participant_data <- Z %*% chol_sigma +
  matrix(means, nrow = n, ncol = p, byrow = TRUE)
```

Justification: `MASS::mvrnorm` uses eigen-decomposition internally, which consumes RNG state differently than the Cholesky approach. For identical seeds, the two methods produce different random draws from the same distribution. While the statistical properties are identical in expectation, exact per-seed reproducibility requires matching the draw method. This also enables sigma caching (pre-compute Cholesky once, reuse across replicates).

Difficulty: Easy. Replace 4 lines. The Cholesky fallback (make.positive.definite on error) should also be copied.

Verification: For the same seed and sigma, both codebases must produce identical participant data matrices.

---

9 Step 6: Match PD handling in build_sigma_matrix

What changes: The 2025 `build_sigma_matrix()` returns NULL for non-PD matrices. Change to match orig: correct via `make.positive.definite()` and continue.

Current (2025):

```
if (!is_pd) {
  return(NULL)
}
```

Target:

```
if (!is_pd) {
  sigma <- corpcor::make.positive.definite(sigma, tol = 1e-3)
}
```

Justification: Returning NULL for non-PD matrices changes the parameter coverage of the simulation. Orig corrects and continues, which means it can run parameter combinations that 2025 rejects. This affects power estimates at the boundary of the PD-feasible region (high c.bm values with OL+BDC designs).

Difficulty: Trivial. Replace the return(NULL) block.

Verification: `build_sigma_matrix()` with high c.bm (e.g. 0.45) must return a valid sigma, not NULL.

---

10 Step 7: Add missing infrastructure functions

What changes: Port `buildtrialdesign`, `censordata`, and `generateSimulatedResults` as tidyverse equivalents.

10.1 7a: build_trial_design (from buildtrialdesign)

The orig function (120 lines) computes tod, tsd, tpb from timepoints, ondrug, and expectancies. The 2025 codebase has `compute_tod_orig`, `compute_tsd_orig`, `compute_tpb_orig` that replicate this logic but are not wired into a design builder.

Approach: Write `build_trial_design()` that wraps the existing compute_*_orig helpers and returns a tibble-based design object with the same structure as orig's output.

Difficulty: Moderate. The logic exists in pieces; needs assembly and testing against orig's output for all four design types (OL, CO, Hybrid, Parallel).

10.2 7b: censor_data (from censordata)

The orig function (90 lines) implements MCAR+biased censoring. Port directly with tibble/dplyr idioms.

Difficulty: Easy. Mostly vectorized math; the data.table eval/parse idioms need replacement with tidy selection.

10.3 7c: generate_simulated_results (from generateSimulatedResults)

The orig function (320 lines) implements the full parameter grid sweep with sigma caching, progressive save, and furrr parallelism. This is the most complex function to port.

Difficulty: High. The nested loop structure with progressive save, parallel dispatch, and closure-based worker functions is intricate. However, the 2025 codebase already has furrr/future infrastructure (unused). The port should follow orig's structure closely.

Justification for all three: Without these functions, the 2025 codebase cannot run a complete simulation. It can only generate single trials and analyze them. The simulation loop, censoring, and trial design builder are required for the full workflow.

---

11 Step 8: Style Reconciliation

11.1 Function name mapping

orig (camelCase)	2025 (snake_case)
`modgompertz`	`mod_gompertz`
`cumulative`	`cumulative` (keep)
`buildSigma`	`build_sigma`
`generateData`	`generate_data`
`buildtrialdesign`	`build_trial_design`
`lme_analysis`	`lme_analysis` (keep)
`censordata`	`censor_data`
`generateSimulatedResults`	`generate_simulated_results`
`validateParameterGrid`	`validate_parameter_grid`
`buildSigma`	`build_sigma`

11.2 Variable name mapping (inside functions)

11.3 Style rules

- Use `|>` (native pipe), not `%>%`
- Use `<-` for assignment
- Use snake_case for all internal variables and function names
- Keep camelCase for external column names that must match orig (`ptID`, `BL`, `Dbc`, `De`)

orig	2025
`nP`	`num_timepoints`
`cl`	`factor_abbrevs` (use `c("tv","pb","br")`)
`dat`	`data`
`modelparam`	`model_param`
`respparam`	`resp_param`
`blparam`	`baseline_param`
`trialdesign`	`trial_design`
`timeptname` / `timeptnames`	`timepoint_name`
`ptID`	`ptID` (keep – external interface)
`bm`, `BL`	`bm`, `BL` (keep – external interface)

- 2-space indentation
- Single quotes for strings

11.4 Estimated style-only changes

Renaming internal variables and converting pipe operators is mechanical but pervasive. Estimate: ~200 line-level changes across the full pm_functions.R file. Low risk since these are find-replace operations that do not affect logic.

Difficulty: Easy but tedious. Best done in a single pass after all functional changes are complete.

---

12 Step 9: Remove dead code from 2025

After alignment, the following 2025-specific code can be removed:

Function / Object	Reason
`mod_gompertz` (old formula)	Replaced by origin-passing version
`calculate_bio_response_with_interaction`	Inline into generate_data (orig does not have this as a separate function)
`calculate_component_halflives`	Returns uniform values; inline the single value
`calculate_carryover` (multi-model)	Only exponential is used; inline the formula
`factor_types` array	Collapse to `factor_abbrevs = c("tv","pb","br")`
`modgompertz_orig`	Redundant after Step 1 replaces mod_gompertz
`compute_tod_orig` / `compute_tsd_orig` / `compute_tpb_orig`	Absorbed into build_trial_design
`build_bm_br_correlations`	Absorbed into build_correlation_matrix
`build_path_sigma`	Redundant after primary pipeline is fixed
`hendrickson_resp_params` / `bl_params` / `corr_params`	Move to analysis scripts as parameter definitions
`validate_correlation_structure`	Diagnostic; keep or move to separate utils
`create_sigma_cache_key`	Replace with orig's approach if needed
`report_parameter_validation`	Diagnostic; keep or move

---

13 Execution Order and Dependencies

```
Step 1 (Gompertz)
  |
  v
Step 2 (Column names) + Step 3 (Category labels)
  |
  v
Step 5 (MVN draw) + Step 6 (PD handling)
  |
  v
Step 4 (lme_analysis)    -- depends on column names being settled
  |
  v
Step 7a (build_trial_design)
  |
  v
Step 7b (censor_data) + Step 7c (generate_simulated_results)
  |
  v
Step 8 (Style cleanup)
  |
  v
Step 9 (Dead code removal)
```

Steps 1–3 can be done in a single pass (all in generate_data and build_sigma_matrix). Steps 5–6 are trivial and can be done alongside. Step 4 is the most complex and should be done carefully with test coverage. Steps 7a–c can proceed in parallel once the core DGP is aligned. Steps 8–9 are cleanup.

---

14 Estimated Effort

---

15 Validation Strategy

After each step, run a comparison test:

1. Build identical parameter inputs (same trial design, model params, resp params, baseline params, seed).

Step	Description	Difficulty	Lines	Time
1	Gompertz fix	Trivial	~5	15 min
2	Column names	Moderate	~50	1 hr
3	Category labels	Easy	~20	30 min
4	lme_analysis rewrite	High	~180	3–4 hr
5	MVN draw method	Easy	~10	15 min
6	PD handling	Trivial	~5	10 min
7a	build_trial_design	Moderate	~80	1.5 hr
7b	censor_data	Easy	~60	1 hr
7c	generate_simulated_results	High	~250	4–5 hr
8	Style cleanup	Easy	~200	1.5 hr
9	Dead code removal	Easy	~100	30 min
–	**Total**			**~14 hr**

2. Call both orig and 2025 functions.
3. Compare outputs numerically.

Level 1 (after Steps 1–3, 5–6): `build_sigma` output must match `buildSigma` output: identical sigma matrix, means vector, and labels.

Level 2 (after Step 5): `generate_data` output must match `generateData` output: identical participant data for same seed.

Level 3 (after Step 4): `lme_analysis` output must match: identical beta, betaSE, p for same input data, across OL, CO, and Hybrid designs.

Level 4 (after Steps 7a–c): Full simulation sweep must match: power estimates within Monte Carlo error bounds for N=200 reps.